Practical: 5
date: 18/09/2026

Write a python program to predict a non-linear target variable y using two predictors x_1 and x_2.

$Y = 3x_1^2+2\sin(x_2) + €$

Where $€ \sim N(0, 2^2)$ is a random noise.

- **Implement Multiple Linear Regressions using Gradient Descent (not using built-in libraries like LinearRegression).**
- **Compare its performance with a nonlinear machine learning model (XGBoost) on the same dataset.**

Procedure:

Step 1: Import the required Python libraries such as NumPy, Pandas, Matplotlib, Scikit-learn, and XGBoost.

Step 2: Set the random seed and generate 500 observations for the predictors x1 and x2 using random values between -5 and 5.

Step 3: Generate the error term ϵ\epsilon from a normal distribution with mean 0 and standard deviation 2.

Step 4: Generate the nonlinear target variable yy using the given formula:

Step 5: Store x1, x2, and y in a Pandas DataFrame and display the first few observations.

Step 6: Divide the dataset into training and testing sets, using 80% for training and 20% for testing.

Step 7: Standardize the predictor variables using the mean and standard deviation of the training data.

Step 8: Add an intercept term and initialize the coefficients to zero for Multiple Linear Regression.

Step 9: Apply the Gradient Descent algorithm to estimate the regression coefficients by repeatedly updating them using the learning rate and calculated gradients. The assignment specifically requires Gradient Descent rather than a built-in LinearRegression model.

Step 10: Use the estimated coefficients to predict the yy values for the test data.

Step 11: Calculate the MSE, RMSE, and R2R^2 score for the Gradient Descent model.

Step 12: Build an XGBoost Regressor model using the same training data and generate predictions for the test data.

Step 13: Calculate the MSE, RMSE, and R2 score for the XGBoost model.

Step 14: Compare the performance of Gradient Descent and XGBoost using the calculated performance measures.

Step 15: Plot the Actual vs Predicted values for both models to visually compare their prediction performance.

Solution:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from xgboost import XGBRegressor

# Generate data
np.random.seed(10)
n = 500
x1 = np.random.uniform(-5, 5, n)
x2 = np.random.uniform(-5, 5, n)
epsilon = np.random.normal(0, 2, n)
y = 3 * x1**2 + 2 * np.sin(x2) + epsilon
data = pd.DataFrame({'x1': x1,'x2': x2,'y': y})
print(data.head())
```

```
         x1        x2          y
0  2.713206 -4.703451  26.278629
1 -4.792481 -4.885715  69.796487
2  1.336482 -1.681560   7.762005
3  2.488039 -4.385689  19.059937
4 -0.014930  0.981735   1.542395
```

```
# Separate predictors and target
X = data[['x1', 'x2']].values
Y = data['y'].values

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=10)

# Standardization
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)
X_train_scaled = (X_train - mean) / std
X_test_scaled = (X_test - mean) / std

# Add intercept
X_train_gd = np.c_[np.ones(X_train_scaled.shape[0]), X_train_scaled]
X_test_gd = np.c_[np.ones(X_test_scaled.shape[0]), X_test_scaled]
```

```
# Gradient Descent
theta = np.zeros(X_train_gd.shape[1])
learning_rate = 0.01
iterations = 5000
m = len(y_train)
for i in range(iterations):
    predictions = X_train_gd @ theta
    error = predictions - y_train
    gradient = (1 / m) * (X_train_gd.T @ error)
    theta = theta - learning_rate * gradient

# Prediction using Gradient Descent
y_pred_gd = X_test_gd @ theta

# Gradient Descent performance
mse_gd = mean_squared_error(y_test, y_pred_gd)
rmse_gd = np.sqrt(mse_gd)
r2_gd = r2_score(y_test, y_pred_gd)
print("\nGradient Descent Results")
print("MSE :", mse_gd)
print("RMSE :", rmse_gd)
print("R2 Score :", r2_gd)
```

```
Gradient Descent Results
MSE : 543.2504788205623
RMSE : 23.307734313325316
R2 Score : 0.015693856947149754
```

```python
# XGBoost model
xgb_model = XGBRegressor(
    n_estimators=100,
    max_depth=3,
    learning_rate=0.05,
    random_state=10)
xgb_model.fit(X_train, y_train)

# Prediction using XGBoost
y_pred_xgb = xgb_model.predict(X_test)
```

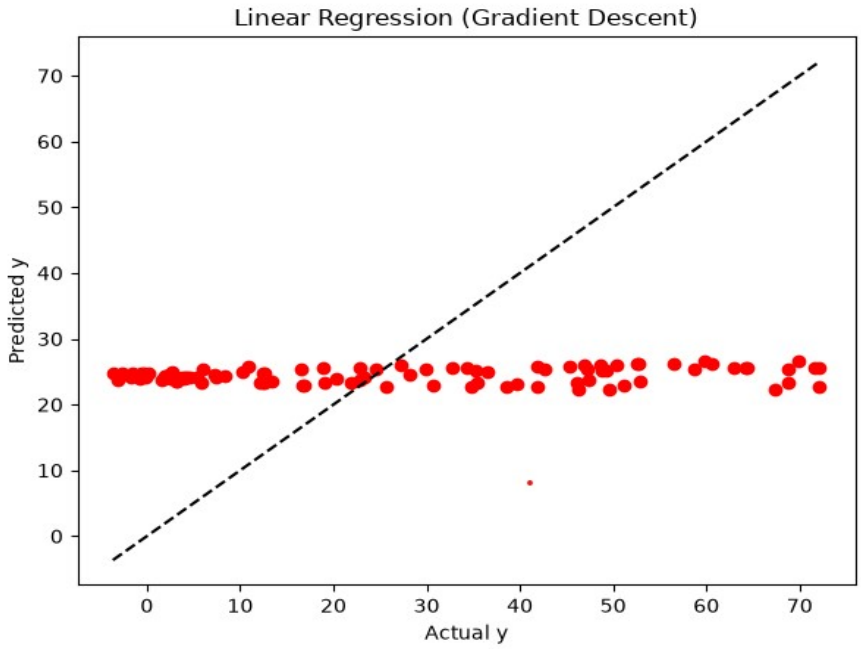
```python
# XGBoost performance
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mse_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)
print("\nXGBoost Results")
print("MSE :", mse_xgb)
print("RMSE :", rmse_xgb)
print("R2 Score :", r2_xgb)
```

```
XGBoost Results
MSE : 6.958696684103197
RMSE : 2.637934169781952
R2 Score : 0.9873916578800354
```

```python
# Comparison
comparison = pd.DataFrame({
    'Model': ['Gradient Descent', 'XGBoost'],
    'MSE': [mse_gd, mse_xgb],
    'RMSE': [rmse_gd, rmse_xgb],
    'R2 Score': [r2_gd, r2_xgb]})
print("\nModel Comparison")
print(comparison)
```

```
Model Comparison
              Model         MSE       RMSE  R2 Score
0  Gradient Descent  543.250479  23.307734  0.015694
1           XGBoost    6.958697   2.637934  0.987392
```

```python
# Plot Actual vs Predicted values
plt.figure(figsize=(12, 5))
# Gradient Descent
plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred_gd, color='red')
plt.plot([y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],'k--')
plt.xlabel('Actual y')
plt.ylabel('Predicted y')
plt.title('Linear Regression (Gradient Descent)')
```



```python
# XGBoost
plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_xgb, color='green')
plt.plot([y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],'k--')
plt.xlabel('Actual y')
plt.ylabel('Predicted y')
plt.title('XGBoost Predictions')
plt.tight_layout()
plt.show()
```

